

TruCheck Help Chatbot

Project guide: how it works and what each file does

How the project works, in plain terms

The chatbot doesn't have the FAQ memorized. Instead, every time someone asks a question, the system looks up the most relevant paragraphs from the FAQ document and hands just those to the AI model, along with an instruction to answer using only that material. This approach is called RAG (Retrieval-Augmented Generation): retrieval means looking things up, generation means the AI writing a natural-sounding answer.

This keeps answers grounded in the real FAQ content rather than the AI guessing or inventing policy, and it makes updates easy - changing the FAQ document and re-running one script is enough to update what the bot knows, with no retraining required.

The three pieces, and why they're separate

Piece	Runs	Job
Ingestion	Once, or whenever the FAQ changes	Reads the FAQ document, splits it into small searchable pieces, and stores them in a local database.
Backend	Continuously (a running server)	Receives a question, searches for the relevant pieces, asks the AI model to answer using them, and sends back the answer.
Frontend	In the user's browser	The chat button and panel the user actually sees and clicks inside the Trucheck portal.

One full round trip

What happens when someone types a question into the chat widget:

- The widget sends the question plus recent conversation history to the backend.
- The backend converts the question into a numeric representation of its meaning (an embedding).
- It searches the stored FAQ pieces for the closest matches by meaning, not exact wording.
- The matching FAQ text is combined with system instructions and sent to the OpenAI model.
- The model writes an answer grounded only in that provided text, plus 2-3 relevant follow-up questions.
- The backend sends the answer and suggestions back; the widget displays the answer bubble and clickable suggestion chips.

File-by-file guide

data/ - the source material

File	What it does
TruCheck_Internal_FAQ.pdf	The FAQ document itself. This is the only thing the bot actually knows. Add more files here to expand what it can answer.

ingestion/ - runs once, to build the searchable knowledge base

File	What it does
extract.py	Opens the PDF or Word file and pulls out the plain text.
chunk.py	Cuts the extracted text into small question-and-answer pieces, and specially repairs tables that get garbled when PDFs are flattened to plain text.
embed_and_store.py	Converts each chunk into a numeric meaning-fingerprint using OpenAI, then saves everything into the ChromaDB database. The only ingestion file that needs the OpenAI API key.
query_test.py	A manual testing tool - type a question, see which chunks come back. Not used by the live chatbot.
requirements.txt	The list of Python packages this folder needs installed.

backend/ - runs continuously, answers questions in real time

File	What it does
main.py	The actual server. Defines the web address (/api/chat) the widget talks to. Started with uvicorn.
schemas.py	Defines the expected shape of a request and a response, keeping the data consistent.
retrieval.py	Converts a question into numbers and asks ChromaDB which stored chunks are closest in meaning.
chat_service.py	Builds the instructions for the AI (FAQ chunks, rules, navigation map), calls OpenAI, and unpacks the answer plus suggested follow-up questions.
nav_map.py	A list of Trucheck's pages and buttons, so the bot can answer navigation questions by pointing to the right screen. Should be edited with the real portal's pages.
requirements.txt	The list of Python packages this folder needs installed.

frontend/ - what the user actually sees and clicks

File	What it does
ChatWidget.jsx	The chat interface itself - the launcher button, message bubbles, input box, and suggestion chips. Controls behavior: what happens on click, type, and send.
ChatWidget.css	All visual styling - colors, spacing, animations, fonts.
public/animations/loading.lottie	The animation shown while waiting for an answer.

Top level

File	What it does
README.md	Written setup instructions for the whole project.

One thing worth remembering

If the bot ever gives a wrong or strange answer, the first question to ask is whether it retrieved the right FAQ chunk - not whether the AI itself is broken. Most problems in a system like this come from search finding the wrong paragraph, not from the AI misbehaving.